

Demo Compose - frontend, backend, dan PostgreSQL

Kasus kerja harian: anggota tim mendapat aplikasi Cloud Notes dan ingin menjalankan frontend, API, serta database dengan konfigurasi yang dapat dibaca dan diulang bersama. Dockerfile menyiapkan image tiap aplikasi; Compose menyatukan cara menjalankan service, network, konfigurasi, dan volume.

Demo ini jembatan dari teori Docker Week 05 ke Lab 06 dan proyek kelompok. Ini **latihan formatif bersama**, tanpa tugas, nilai, atau penyerahan terpisah. Proyek tetap kelompok **3 mahasiswa**, dengan presentasi minggu **7 (UTS)** dan **14 (UAS)**. Semua pertanyaan diskusi beserta kunci tersedia di bawah.

1. Dua contoh yang sengaja berbeda

Contoh	Port host	Penyimpanan	Kegunaan belajar
API utama Lab 05, cloud-notes-api:theory-online	8000	Dict Python dalam memori	Dockerfile, build/run, validasi, dan data hilang saat proses restart.
API Compose di examples/compose-notes	13005 → container 8000	PostgreSQL	Backend berkomunikasi dengan database melalui DNS service.
Frontend Compose, service web	13006 → container 80	Menggunakan API	Browser membuat dan membaca catatan melalui Nginx.
Database Compose, service db	Tidak dipublikasikan ke host	Named volume	PostgreSQL menerima koneksi internal pada port 5432.

Kode Compose adalah contoh terpisah. Dockerfile/API memori Lab 05 tetap tersedia. Image yang diperiksa pada demo Scout utama adalah image API memori, sehingga jangan menyamakan nama tag atau hasil scannya dengan image backend Compose.

Alamat pada screenshot online: pengujian Codespaces kelas memakai API **13105** dan web **13106**, karena port 13005 sudah dipakai proses lain yang dipertahankan. Nilai default repo tetap API 13005/web 13006. Port di dalam container tidak berubah: API 8000, web 80, database 5432. Bagian berikut menunjukkan cara memilih port; URL browser dan curl harus mengikuti pilihan yang sama.

Alur request: browser membuka frontend → request relatif [/api/notes](#) → Nginx meneruskan ke [api:8000](#) → FastAPI membaca/menulis PostgreSQL pada [db:5432](#). Nama [api](#) dan [db](#) ditemukan melalui DNS network Compose, bukan alamat IP yang harus dihafal. Database tidak memerlukan forwarding publik. [Networking Compose](#)

2. Persiapan: browser atau mesin lokal

Jalur online: buka repo [meet5CloudService](#), pilih Code → Codespaces, gunakan konfigurasi repo dan mesin 2 core. Tunggu terminal Linux serta daemon Docker siap. Laptop hanya memerlukan browser, akun GitHub, internet, dan kuota; Docker berjalan di mesin cloud. Waktu provisioning/unduhan di luar waktu show kelas. Jangan memakai editor ringan [github.dev](#) sebagai pengganti Codespaces.

2.1 Mulai dari komputer kampus atau laptop yang belum memiliki repo

Prasyarat komputer kampus: Git dan Docker Desktop/Engine sudah dipasang serta diizinkan oleh pengelola laboratorium. Buka Docker Desktop dan tunggu Engine siap. Mahasiswa menyiapkan salinan kode di folder milik pengguna sendiri, misalnya **Documents/CloudServices**. Langkah ini tidak memerlukan hak admin mahasiswa.

Jalankan empat pemeriksaan berikut; command sama di PowerShell, Git Bash, atau terminal Linux:

```
git --version
docker version
docker compose version
docker info --format '{{.OSType}}'
```

Baca hasil: Git menampilkan versinya; `docker version` harus menampilkan bagian **Client dan Server**. Client tersedia tetapi Server gagal berarti Engine belum siap. `docker compose version` memastikan Compose tersedia; `docker info` di atas harus menghasilkan **linux**, karena image demo memakai Linux containers. Jika `git` atau `docker` tidak ditemukan, atau Engine tidak dapat dimulai, minta pengelola lab menyiapkan komputer sebelum melanjutkan.

Pilihan A: PowerShell pada Windows

Jalankan **sekali untuk salinan baru**. `MyDocuments` mengikuti lokasi Documents pengguna, termasuk jika kampus mengalihkannya ke OneDrive.

```
$labWorkspace = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'CloudServices'
New-Item -ItemType Directory -Path $labWorkspace -Force | Out-Null
Set-Location -LiteralPath $labWorkspace
git clone https://github.com/SeedFlora/meet5CloudService.git
Set-Location -LiteralPath meet5CloudService
Get-Location
git remote -v
Test-Path -LiteralPath examples/compose-notes/compose.yaml
Test-Path -LiteralPath examples/compose-notes/.env.example
```

Fungsi dan cara kerja: dua baris pertama menyiapkan folder yang dapat ditulis oleh pengguna. `git clone` mengunduh isi repo sekaligus riwayat Git dan remote `origin`. `Set-Location meet5CloudService` masuk ke **root repo**, bukan folder contoh Compose. `Get-Location` menunjukkan lokasi aktif, `git remote -v` harus menunjuk `SeedFlora/meet5CloudService`, dan dua `Test-Path` harus menghasilkan **True, True**. Salinan ini dipakai untuk menjalankan demo; mahasiswa tidak perlu mengirim perubahan ke repo pengajar.

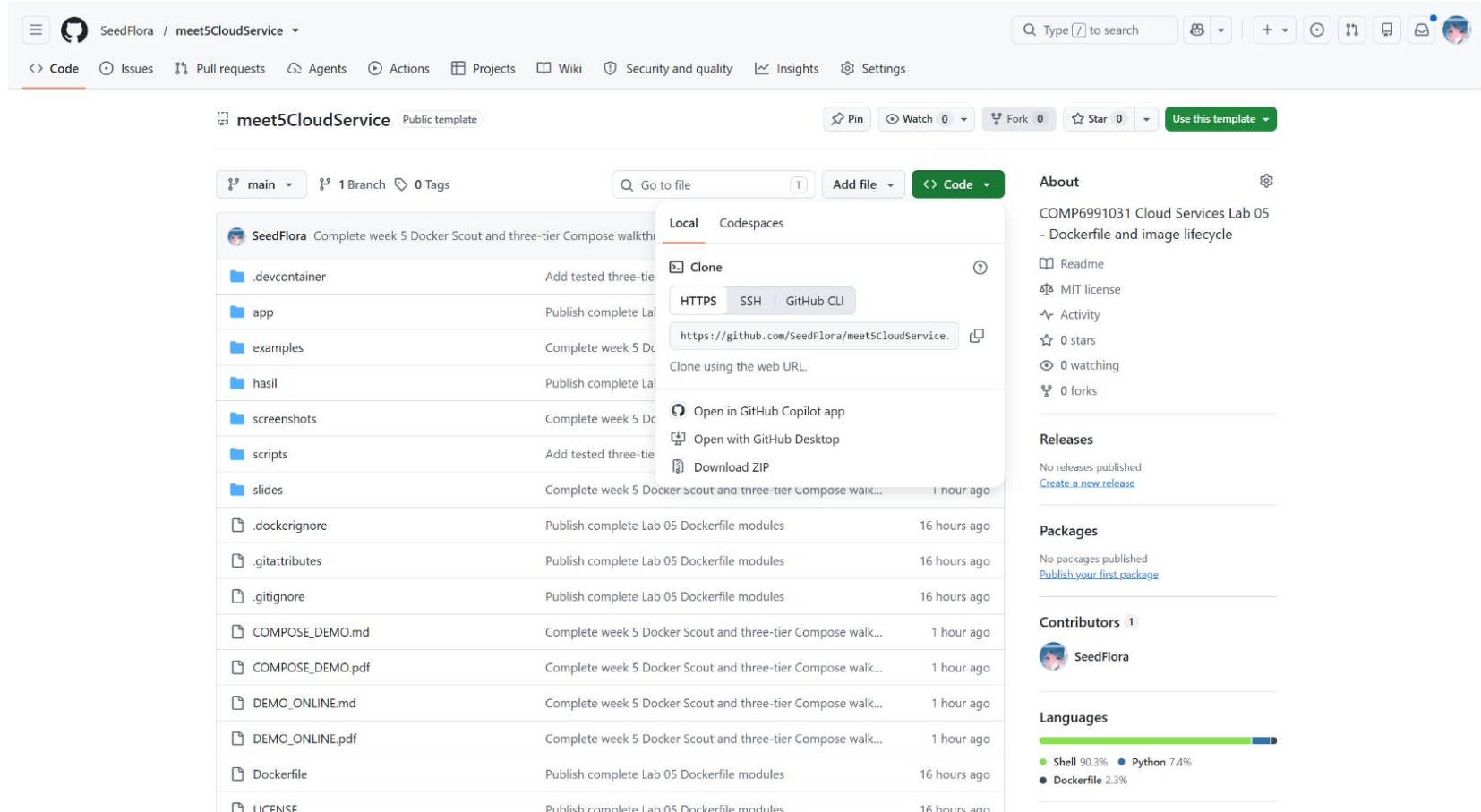
Pilihan B: Git Bash pada Windows atau Bash pada Linux/WSL

```
mkdir -p "$HOME/Documents/CloudServices"
cd "$HOME/Documents/CloudServices"
git clone https://github.com/SeedFlora/meet5CloudService.git
cd meet5CloudService
pwd
git remote -v
ls -l examples/compose-notes/compose.yaml examples/compose-notes/.env.example
```

Fungsi dan cara kerja: `mkdir -p` membuat folder kerja pengguna jika belum ada; `cd` berpindah lokasi. `pwd` harus berakhir pada **meet5CloudService**, sedangkan `ls -l` harus menampilkan kedua file contoh. `$HOME` adalah folder pengguna terminal tersebut. Pada WSL, Documents di sini berada pada filesystem pengguna Linux; pada Git Bash, ia berada pada profil pengguna Windows. Untuk mengikuti command Bash pada bagian selanjutnya di Windows, gunakan **Git Bash** pada lokasi salinan repo yang dipilih.

Gambar 1. GitHub menyediakan URL HTTPS untuk menyalin repo Lab 05 ke komputer kampus

```
git clone https://github.com/SeedFlora/meet5CloudService.git
```



Command / langkah: buka Code → Local → HTTPS, salin URL, lalu jalankan `git clone https://github.com/SeedFlora/meet5CloudService.git` pada terminal di folder kerja pengguna. Fungsi: mengambil salinan sumber praktikum yang sama untuk seluruh kelas. Cara kerja: GitHub menyediakan URL HTTPS; Git mengunduh isi dan riwayat repo lalu membuat folder `meet5CloudService`. Baca: screenshot menunjukkan pilihan URL pada GitHub, sedangkan keberhasilan clone diperiksa di terminal melalui remote `origin` dan keberadaan `examples/compose-notes/compose.yaml` serta `.env.example`. Screenshot ini belum membuktikan container berjalan.

Jika salinan repo sudah ada

Buka terminal dari **root salinan meet5CloudService yang sudah ada**, lalu periksa:

```
git remote -v
git status --short
```

Jika **origin** cocok dengan repo di atas dan **git status --short** **tidak menampilkan apa pun**, perbarui dengan:

```
git pull --ff-only origin main
```

Kunci: pull hanya dilakukan pada salinan yang bersih; jangan menimpa perubahan latihan yang belum disimpan. Jika ada perubahan, pertahankan salinan itu dan diskusikan pembaruannya bersama dosen. Jika clone melaporkan *destination path already exists*, gunakan pemeriksaan salinan yang sudah ada ini. Clone tidak perlu diulang di dalam repo. Pada Codespaces, repo sudah tersedia setelah provisioning, sehingga langsung lanjut dari root repo.

Kebutuhan internet: clone dan build pertama memerlukan internet untuk mengambil kode, image dasar, serta dependency aplikasi. Jalur lokal berarti Docker berjalan pada komputer kampus/laptop; ini bukan jaminan seluruh persiapan bisa offline. Pengajar perlu menyiapkan image dan dependency sebelum sesi bila jaringan kelas terbatas. Docker Scout tetap memakai layanan online pada langkah scan opsional.

2.2 Masuk folder contoh Compose

Jalur lokal: gunakan Docker Desktop/Engine yang sudah berjalan. Port **13005** dan **13006** harus kosong. Untuk command Bash, gunakan terminal Linux/WSL atau Git Bash; perintah Docker Compose juga dapat dijalankan di PowerShell.

Dari root repo:

```
docker version
docker compose version
cd examples/compose-notes
```

Seluruh command Compose berikut dijalankan dari folder **examples/compose-notes**, supaya Compose membaca **compose.yaml** dan **.env** yang benar.

Padanan pindah folder pada PowerShell: **Set-Location -LiteralPath examples/compose-notes**.

Sebelum melanjutkan, **Get-Location** atau **pwd** harus berakhir pada

meet5CloudService/examples/compose-notes. Pemeriksaan file pada 2.1 dilakukan dari root repo; salin **.env** pada bagian berikut setelah berada di folder contoh ini.

Salin konfigurasi khusus kelas

Bash:

```
cp .env.example .env
```

PowerShell:

```
Copy-Item -LiteralPath .env.example -Destination .env
```

File contoh memakai database `cloudnotes`, user `clouduser`, dan password `classroom-demo-only-change-me`. Password ini sengaja diketahui semua orang, hanya untuk data dummy pada demo terisolasi; jangan dipakai pada sistem kerja atau layanan produksi. `.env` diabaikan Git. Ganti secret pada sistem nyata melalui mekanisme secret yang sesuai, lalu hindari menampilkan nilainya pada presentasi atau commit. `docker compose config` dapat menampilkan nilai environment yang sudah disubstitusi; gunakan `--quiet` untuk pemeriksaan syntax tanpa mencetak semuanya.

Pilih port sebelum menjalankan service

Default lokal memakai API **13005** dan web **13006**. Untuk mengikuti screenshot online atau menghindari benturan port, jalankan di terminal Linux setelah menyalin `.env`:

```
sed -i 's/^API_PORT=.*API_PORT=13105/; s/^WEB_PORT=.*WEB_PORT=13106/' .env
```

Fungsi: mengganti baris `API_PORT` dan `WEB_PORT` yang memang sudah ada pada file contoh. Tidak mengubah port aplikasi di dalam container atau mematikan proses lain. Jalankan sebelum `up`; jika service sendiri sudah dibuat pada port lama, recreate service melalui `docker compose up -d --build --wait` setelah konfigurasi diperbarui. Jangan mengubah port sambil tetap menggunakan URL lama.

Agar command `curl` pada panduan mengikuti port yang dipilih, baca nilainya dari `.env` pada **terminal Bash yang sama**:

```
API_HOST_PORT="$(sed -n 's/^API_PORT=//p' .env | tail -n 1)"
WEB_HOST_PORT="$(sed -n 's/^WEB_PORT=//p' .env | tail -n 1)"
printf 'Port host API=%s; frontend=%s\n' "$API_HOST_PORT" "$WEB_HOST_PORT"
```

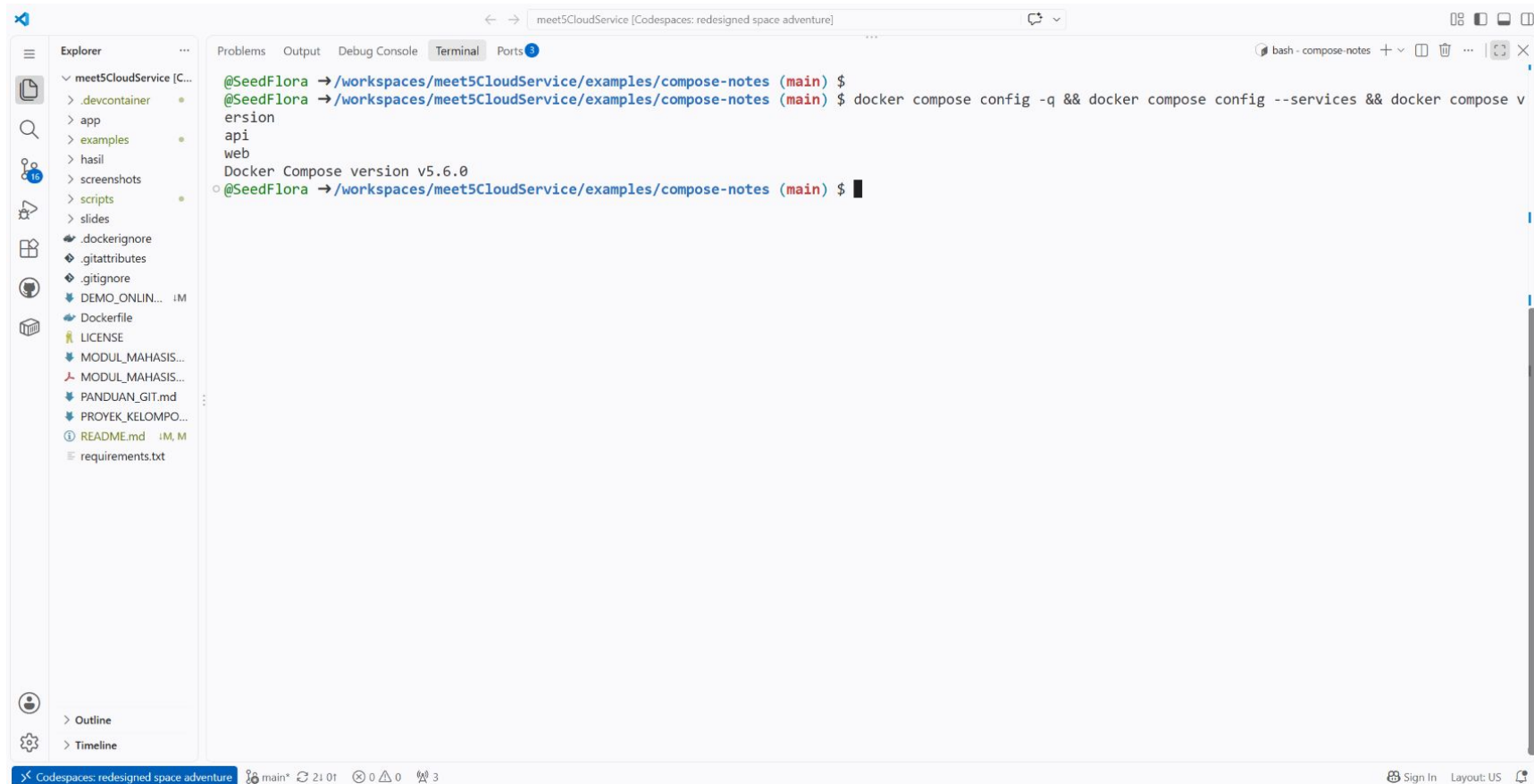
File contoh menyediakan kedua baris port sebagai angka tanpa quote. Jika membuka terminal baru, jalankan tiga baris tersebut lagi. Variabel Bash ini hanya membantu command `curl`; mapping Docker ditentukan oleh `.env`, bukan oleh dua variabel tersebut. Pengguna PowerShell dapat memakai URL literal sesuai `.env`, misalnya `curl.exe -i http://127.0.0.1:13105/health` untuk alternatif online.

```
docker compose config --quiet
```

Fungsi: memeriksa apakah Compose dapat membaca model dan variabelnya. **Cara membaca:** exit sukses tanpa pesan bukan berarti aplikasi sudah berjalan; ini baru pemeriksaan konfigurasi. `docker compose config --services` dapat menunjukkan nama `web`, `api`, dan `db` tanpa menampilkan password.

Gambar 2. Pemeriksaan konfigurasi Compose dan versi pada terminal Codespaces

```
docker compose config -q  
docker compose config --services  
docker compose version
```



Screenshot aktual 8 Oktober 2026. Command: `docker compose config -q`, `docker compose config --services`, dan `docker compose version`. Fungsi: memvalidasi model tanpa mencetak secret, mengenali nama service, serta versi Compose. Config berhasil kembali ke prompt; ini belum menjalankan aplikasi. Versi pada uji ini v5.6.0; versi pada mesin lain dapat berbeda.

3. Build dan jalankan tiga layanan

```
docker compose up -d --build --wait
```

- **up**: membuat dan menjalankan service sesuai model Compose.
- **--build**: membangun image aplikasi sebelum memulai service yang memerlukannya.
- **-d**: menjalankan service di background agar terminal kembali menerima command.
- **--wait**: menunggu service berstatus running/healthy sesuai healthcheck yang tersedia. Ini tetap perlu dilengkapi pengujian HTTP, penulisan, serta pembacaan catatan.

Compose memiliki healthcheck untuk database/API/web dan dependency kesiapan agar startup tidak hanya bergantung pada urutan proses dibuat. Aplikasi tetap perlu menangani kegagalan koneksi selama operasional. [Startup order](#), [Compose up](#)

Gambar 3. Compose membangun image dan menunggu database, API, dan web healthy

```
docker compose up -d --build --wait
.env
```

```
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $ cp .env.example .env && printf '
API_PORT=13105
WEB_PORT=13106
' >> .env && docker compose up -d --build --wait
=> CACHED [web 3/3] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> [web] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:df2d8dbe62e35d061a66df7660a04293e9964869b29c7fb40b72264c4ea6de94 0.0s
=> => exporting config sha256:cf56c24278432b6a261f1a886bdc2440c1ce056294a6c571445645b016e53272 0.0s
=> => exporting attestation manifest sha256:7f137987c8fb163221f1dce9b0dce4021a8aab7c4e0c94516ea7991e54801329 0.0s
=> => exporting manifest list sha256:49dfdac2c37c17736aa8a984a56a36f3c581dc6e6e052b4fb23e5ee0621c6c 0.0s
=> => naming to docker.io/library/cloudlab05-compose-web:latest 0.0s
=> => unpacking to docker.io/library/cloudlab05-compose-web:latest 0.0s
=> CACHED [api runtime 2/5] RUN useradd --create-home --uid 10001 appuser 0.0s
=> CACHED [api runtime 3/5] WORKDIR /app 0.0s
=> CACHED [api builder 2/4] WORKDIR /build 0.0s
=> CACHED [api builder 3/4] COPY requirements.txt . 0.0s
=> CACHED [api builder 4/4] RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [api runtime 4/5] COPY --from=builder /opt/venv /opt/venv 0.0s
=> CACHED [api runtime 5/5] COPY app ./app 0.0s
=> [api] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:d9119f5b7f5dc80c7b19391c74fb7f4f00ac8e15b068321419a993e203bea838 0.0s
=> => exporting config sha256:632c9db9992cfff2a4ffff9bec351f3984ef3cad80b990382613c157eed4e9e3e 0.0s
=> => exporting attestation manifest sha256:12a52cc1328e59981c6c74dfa49dc6c87ddbf8471cc57ed0b357d10672863fbd 0.0s
=> => exporting manifest list sha256:2df7aec8b10f0e9376c72d76fae5f8b603700978faad9626b7242d1596b08b18 0.0s
=> => naming to docker.io/library/cloudlab05-compose-api:latest 0.0s
=> => unpacking to docker.io/library/cloudlab05-compose-api:latest 0.0s
=> [web] resolving provenance for metadata file 0.0s
=> [api] resolving provenance for metadata file 0.0s
[+] up 5/5
✓ Image cloudlab05-compose-api Built 1.5s
✓ Image cloudlab05-compose-web Built 1.5s
✓ Container cloudlab05-compose-db-1 Healthy 7.6s
✓ Container cloudlab05-compose-api-1 Healthy 7.6s
✓ Container cloudlab05-compose-web-1 Healthy 12.0s
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $
```

Command utama: `docker compose up -d --build --wait`, setelah memilih `API_PORT=13105` dan `WEB_PORT=13106` dalam `.env`. Fungsi: membuild image API/web dan menjalankan tiga service. Baca hasil: image Built serta container db/api/web Healthy. Screenshot memakai penambahan nilai port pada akhir `.env`; panduan menggunakan `sed` untuk mengganti baris yang sudah ada agar file lebih mudah dibaca. Kedua alur memilih port host alternatif yang sama.

4. Baca status, log, dan batas network

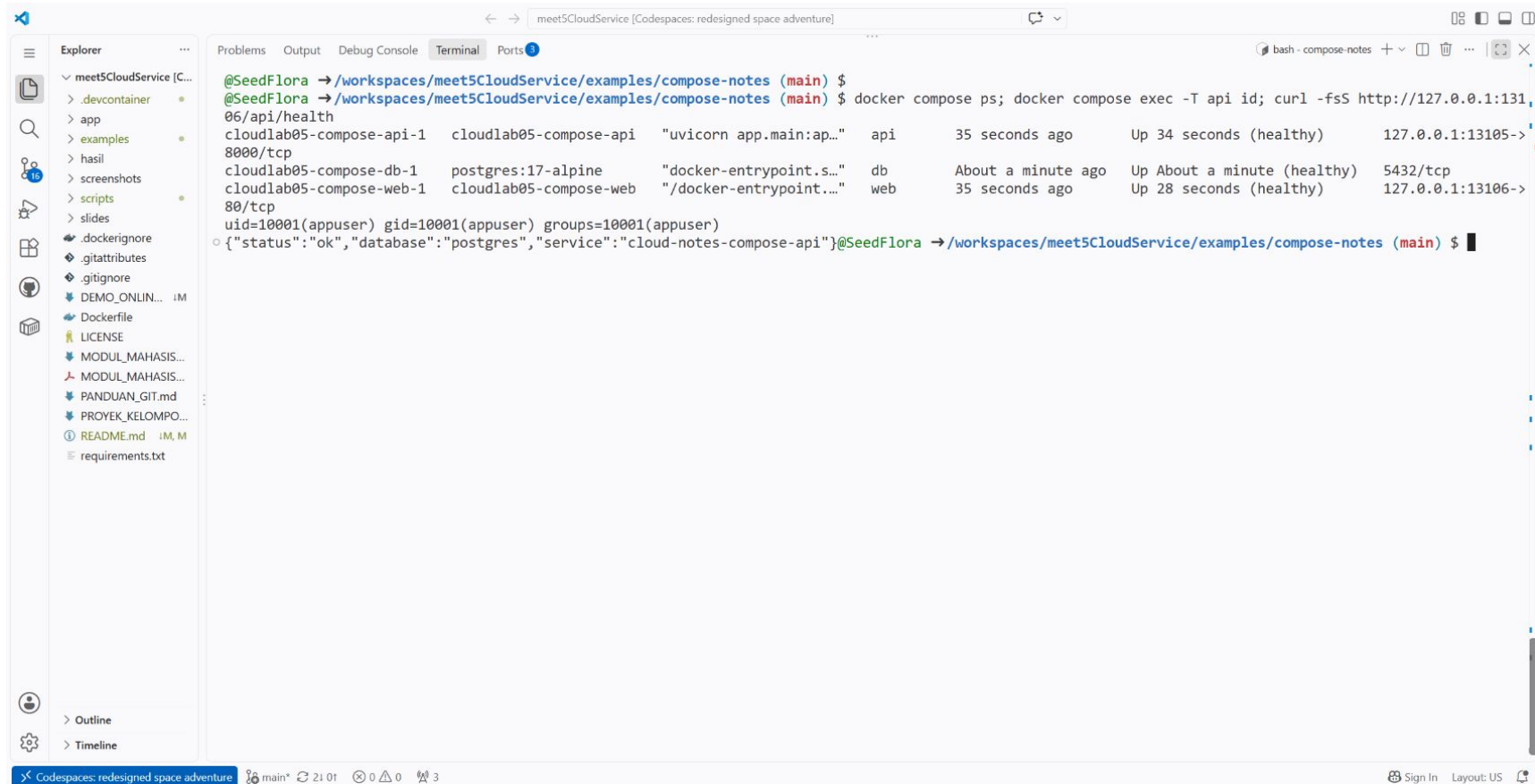
```
docker compose ps
docker compose logs --tail 20 api
docker compose logs --tail 20 db
docker compose logs --tail 20 web
docker compose exec -T api id
```

Yang diperiksa: tiga service berjalan, status healthcheck yang ditampilkan, publish API **13005:8000** dan web **13006:80** secara default, atau **13105:8000/13106:80** untuk alternatif screenshot, serta tidak ada mapping port database ke host. Mapping API/web memakai loopback host **127.0.0.1**; Codespaces kemudian meneruskannya melalui tab Ports. PostgreSQL dapat menampilkan **5432/tcp** pada kolom port internal; itu berbeda dari **0.0.0.0:5432->5432/tcp**. User proses API adalah UID **10001**, sehingga aplikasi tidak dijalankan sebagai root.

Log menjelaskan aktivitas tiap service: API menerima request, database siap menerima koneksi, dan Nginx melayani frontend/proxy. Status **Up** saja belum membuktikan alur simpan/baca bekerja. [Model aplikasi Compose](#)

Gambar 4. Status tiga service, UID 10001, dan respons health dari backend PostgreSQL

```
docker compose ps
docker compose exec -T api id
curl -fsS http://127.0.0.1:13106/api/health
```



Command: `docker compose ps`, `docker compose exec -T api id`, dan `curl -fsS http://127.0.0.1:13106/api/health`. Fungsi: melihat mapping dan user serta menguji request melalui frontend/proxy. Baca hasil: API 13105→8000, web 13106→80, db hanya 5432/tcp internal, UID 10001, serta status ok/database postgres. API `/health` menjalankan `SELECT 1` ke database sebelum memberi respons ok.

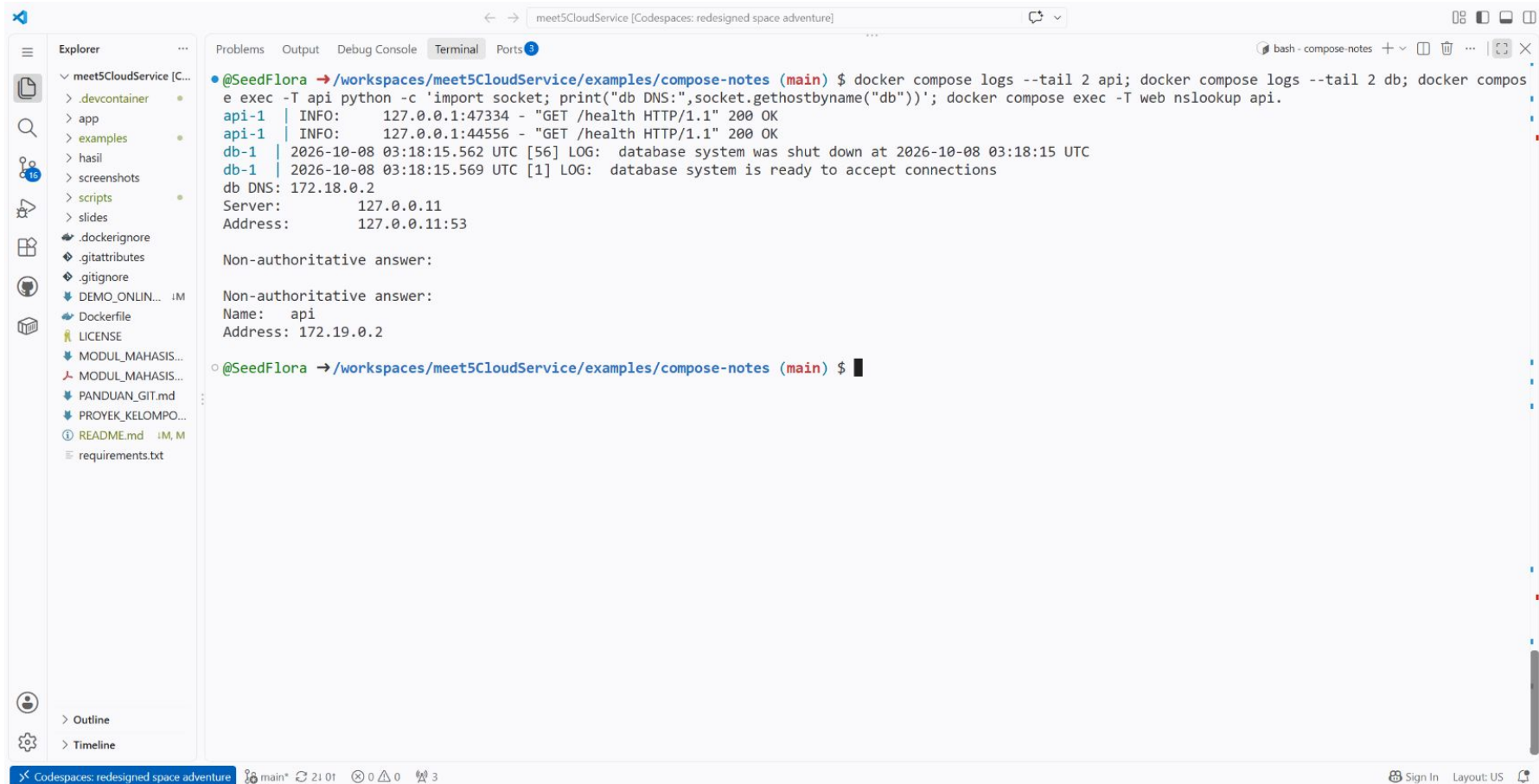
DNS service: nama lebih stabil daripada IP

```
docker compose exec -T api python -c 'import socket; print("db DNS:", socket.gethostbyname("db"))'
docker compose exec -T web nslookup api.
```

Fungsi: API meresolusikan nama database `db`; web meresolusikan nama backend `api`. Python dijalankan **di container API**, bukan mensyaratkan Python pada laptop/host. `-T` menonaktifkan alokasi terminal interaktif untuk command singkat. Titik akhir pada `api`. membuat query DNS absolut sehingga `nslookup` tidak mencoba menambahkan search suffix milik lingkungan cloud. Konfigurasi aplikasi tetap memakai nama service `api` dan `db`; jangan menggantinya dengan IP screenshot.

Gambar 5. Log API/database dan hasil DNS internal Compose

```
docker compose logs --tail 2 api
docker compose logs --tail 2 db
```



The screenshot shows a VS Code interface with a terminal window open. The terminal displays the output of the following commands:

```
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $ docker compose logs --tail 2 api; docker compose logs --tail 2 db; docker compose exec -T api python -c 'import socket; print("db DNS:", socket.gethostbyname("db"))'; docker compose exec -T web nslookup api.
```

The output shows logs for the 'api' and 'db' services, followed by the results of a DNS lookup for 'api'.

```
api-1 | INFO:      127.0.0.1:47334 - "GET /health HTTP/1.1" 200 OK
api-1 | INFO:      127.0.0.1:44556 - "GET /health HTTP/1.1" 200 OK
db-1   | 2026-10-08 03:18:15.562 UTC [56] LOG:  database system was shut down at 2026-10-08 03:18:15 UTC
db-1   | 2026-10-08 03:18:15.569 UTC [1] LOG:  database system is ready to accept connections
db DNS: 172.18.0.2
Server:      127.0.0.11
Address:     127.0.0.11:53

Non-authoritative answer:

Non-authoritative answer:
Name:   api
Address: 172.19.0.2
```

The terminal window also shows the file explorer on the left with a project structure including folders like 'app', 'examples', 'hasil', 'screenshots', 'scripts', 'slides', and files like 'LICENSE', 'README.md', and 'requirements.txt'.

Command: `docker compose logs --tail 2 api`, `docker compose logs --tail 2 db`, lalu dua command DNS di atas. Fungsi: menghubungkan kesiapan layanan dengan komunikasi antarcontainer. Pada snapshot, API menemukan db pada 172.18.0.2 dan web menemukan api pada 172.19.0.2; IP bukan nilai tetap dan dapat berubah ketika service/network dibuat kembali. Server DNS 127.0.0.11 adalah resolver embedded Docker pada network container ini.

5. Tampilkan frontend dan Swagger

Di Codespaces: buka tab Ports, forward port host yang dipilih dalam `.env`: **13006/13005** secara default atau **13106/13105** pada screenshot online. Tambahkan port secara manual bila belum terlihat, pertahankan visibility **Private**, lalu Open in Browser. Gunakan URL forwarding yang benar-benar ditampilkan, tanpa menebak nama Codespace.

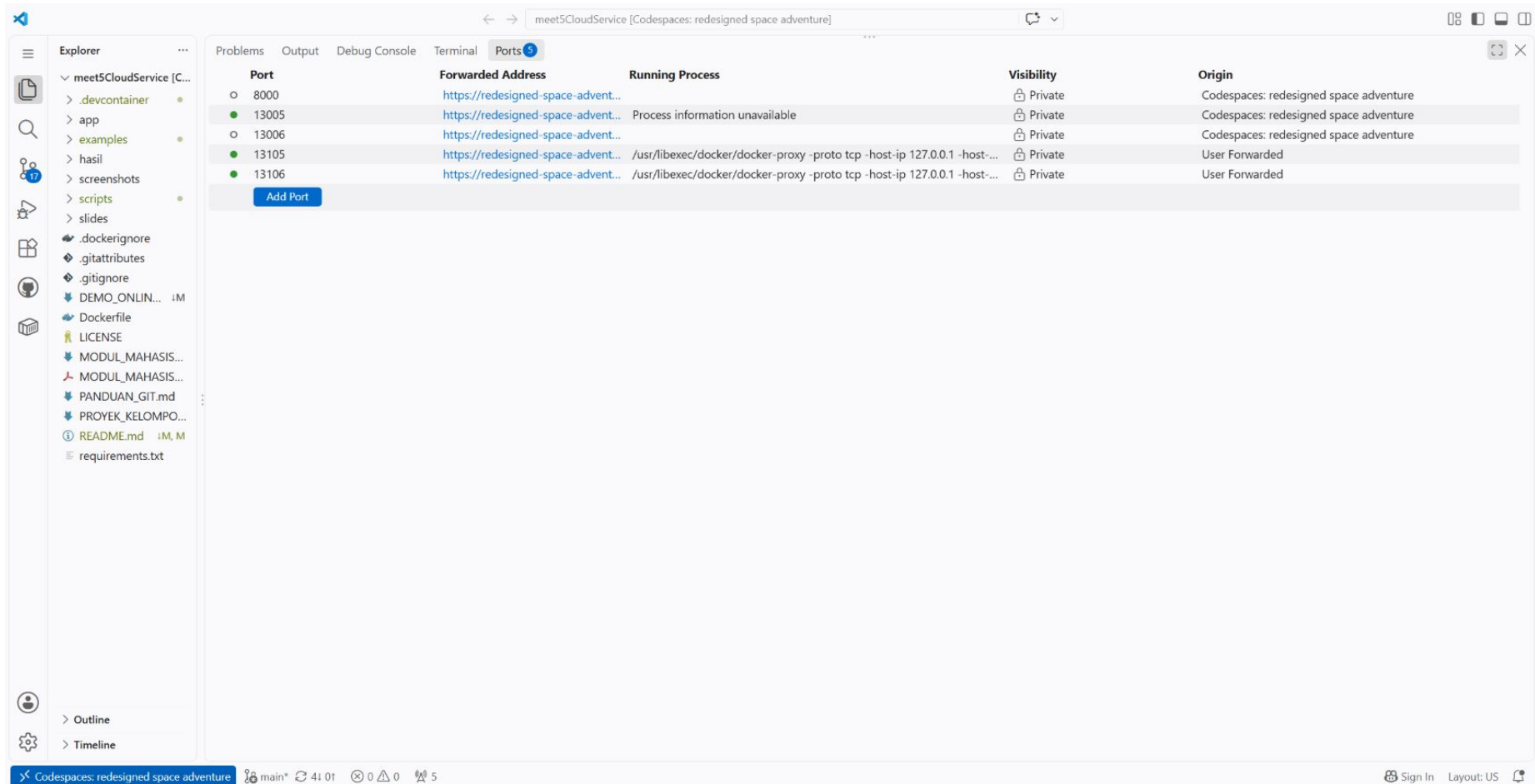
- **Frontend:** URL port 13006, path `/`.
- **Swagger:** URL port 13005, tambahkan `/docs`.
- **Di laptop lokal:** `http://127.0.0.1:13006/` dan `http://127.0.0.1:13005/docs`.

Untuk alternatif screenshot, frontend memakai port **13106** dan Swagger/API **13105**. Browser Codespaces memakai URL forwarding port tersebut; jangan mengetik localhost laptop.

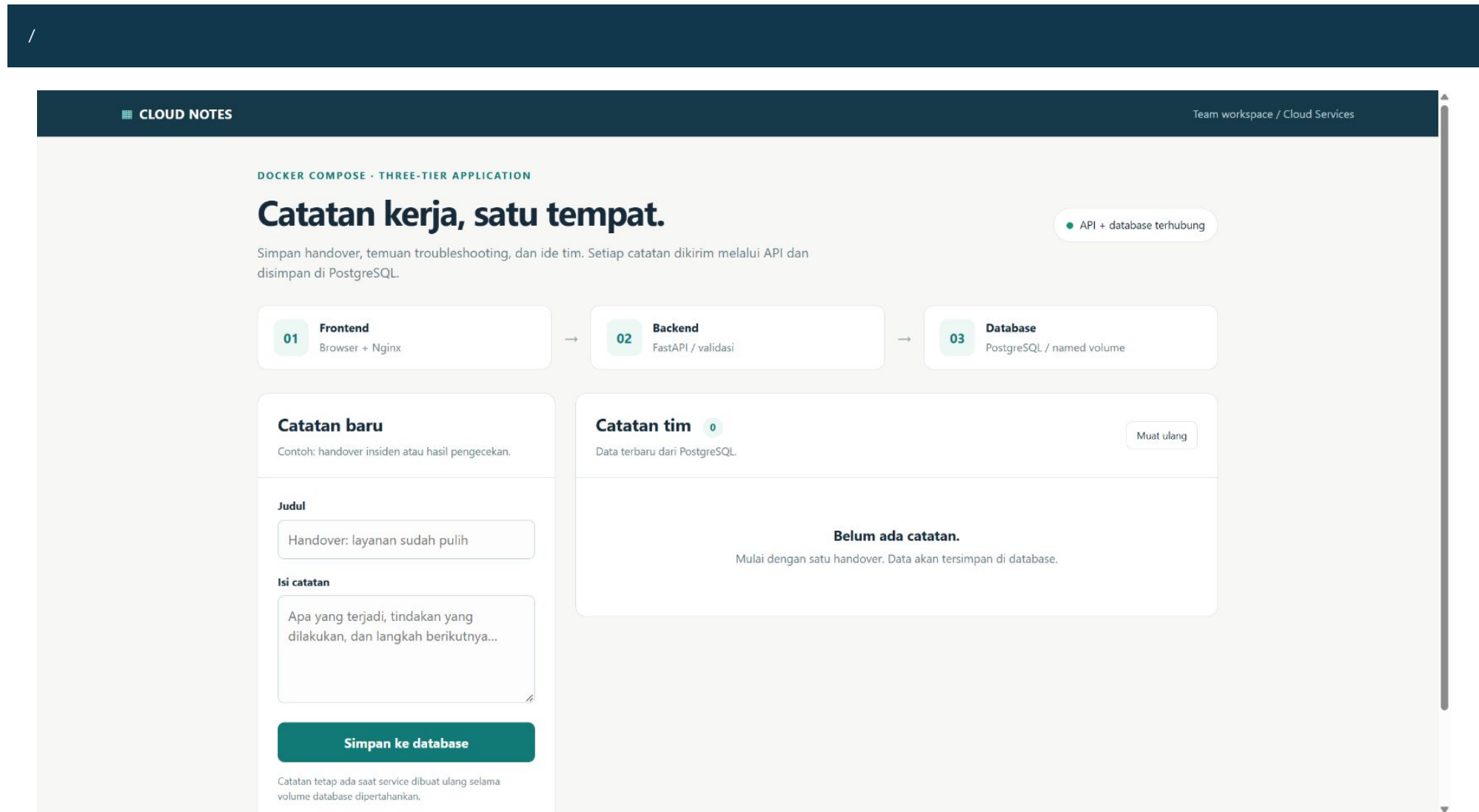
Pada terminal Codespaces, `127.0.0.1` menunjuk mesin cloud. Pada browser laptop, alamat yang sama menunjuk laptop, sehingga akses online harus menggunakan URL tab Ports. Request frontend memakai path relatif `/api/`; browser tidak perlu memanggil alamat internal `api:8000` atau `db:5432`. Nginx di dalam network Compose yang meneruskan request ke backend.

Gambar 6. Port forwarding private untuk API 13105 dan frontend 13106

Ports → Add Port 13105 dan 13106 → Visibility Private → Open in Browser.



Langkah: Ports → Add Port 13105 dan 13106 → Visibility Private → Open in Browser. Fungsi: menghubungkan browser laptop dengan port host Codespaces. Baca hasil: kedua port alternatif diteruskan, sementara port 13005 yang sudah dipakai proses lain tetap ada. Tidak ada forwarding port database. URL forwarding ini lapisan tambahan setelah publish port Docker.

Gambar 7. Frontend Cloud Notes dibuka melalui browser dan belum memiliki catatan

Langkah: buka URL port 13106 dari tab Ports, path /. Fungsi: menampilkan frontend dan koneksi ke backend/database. Baca hasil: indikator API + database terhubung dan daftar belum ada catatan. Indikator berasal dari request `/api/health`; isi daftar berasal dari PostgreSQL melalui API, bukan data palsu dalam HTML.

6. Simpan dan baca catatan end to end

Pada halaman frontend, isi judul **Demo Compose** kelas dan isi **Data ini disimpan pada PostgreSQL.**, lalu gunakan tombol penyimpanan. Catatan harus muncul pada daftar setelah request berhasil. Tampilkan Swagger **/health** dan **/runtime** untuk menghubungkan antarmuka dengan backend serta jenis storage.

Padanan pengujian pada terminal **Bash/Linux**; pada PowerShell gunakan **curl.exe** untuk memilih curl asli:

```
curl -i "http://127.0.0.1:${API_HOST_PORT}/health"
curl -i "http://127.0.0.1:${API_HOST_PORT}/runtime"
curl -i -X POST "http://127.0.0.1:${WEB_HOST_PORT}/api/notes" \
  -H 'Content-Type: application/json' \
  -d \
    '{"title":"Demo Compose terminal","content":"Browser dan terminal menggunakan backend yang sama."}'
curl -i "http://127.0.0.1:${WEB_HOST_PORT}/api/notes"
curl -i "http://127.0.0.1:${API_HOST_PORT}/notes"
```

Fungsi: GET health/runtime memeriksa API dan identitas storage; POST melalui port web menguji proxy serta penyimpanan; GET melalui web dan port API menguji pembacaan backend yang sama. **Hasil yang diharapkan:** HTTP **200** untuk GET, HTTP **201** untuk pembuatan catatan, **/runtime** memakai **storage: postgres** dan host database **db**, serta catatan yang dibuat ada di daftar. ID dan urutan catatan dapat berbeda. Periksa isi respons, bukan status saja.

Validasi tambahan bersama dosen:

```
curl -i -X POST "http://127.0.0.1:${WEB_HOST_PORT}/api/notes" \
  -H 'Content-Type: application/json' \
  -d '{"title":"","content":"Contoh input ditolak."}'
curl -i "http://127.0.0.1:${API_HOST_PORT}/notes/0"
```

Kunci: judul spasi saja harus ditolak **422**; ID 0 yang tidak dibuat aplikasi menghasilkan **404**. Itu respons validasi/route yang diharapkan, bukan tanda container rusak. Backend juga menyediakan DELETE **/notes/{id}** dengan respons **204** untuk ID yang ada; penghapusan data tidak diperlukan pada demonstrasi persistensi di bawah.

Gambar 8. Catatan handover dibuat dari frontend dan ditampilkan dari PostgreSQL

isi form lalu klik Simpan ke database pada frontend port 13106.

The screenshot displays the 'CLOUD NOTES' application interface. At the top, a dark header bar contains the logo and 'CLOUD NOTES' on the left, and 'Team workspace / Cloud Services' on the right. Below the header, a section titled 'DOCKER COMPOSE · THREE-TIER APPLICATION' features the main heading 'Catatan kerja, satu tempat.' and a subtext: 'Simpan handover, temuan troubleshooting, dan ide tim. Setiap catatan dikirim melalui API dan disimpan di PostgreSQL.' A status indicator shows 'API + database terhubung'.

A diagram illustrates the three-tier architecture:

- 01 Frontend**: Browser + Nginx
- 02 Backend**: FastAPI / validasi
- 03 Database**: PostgreSQL / named volume

The interface is divided into two main sections:

- Catatan baru**: A form for creating new notes. It includes a title field (e.g., 'Handover: layanan sudah pulih'), a text area for the note content (e.g., 'Apa yang terjadi, tindakan yang dilakukan, dan langkah berikutnya...'), and a 'Simpan ke database' button. A note at the bottom states: 'Catatan tetap ada saat service dibuat ulang selama volume database dipertahankan.'
- Catatan tim 1**: A section displaying data from PostgreSQL. It shows a 'Muat ulang' button and a list of notes. The first note is titled 'Handover: layanan checkout pulih' and contains the text: 'Tim mengganti konfigurasi koneksi database, memeriksa /health, lalu mencatat langkah pemulihan untuk shift berikutnya. Data ini contoh untuk kelas.' It also includes a timestamp: '#1 - 8/10/2026, 10:21:58' and a 'Hapus' button.

Langkah: isi form lalu klik Simpan ke database pada frontend port 13106. Cara kerja: JavaScript POST `/api/notes`; Nginx meneruskan ke FastAPI; backend INSERT ke PostgreSQL; frontend GET daftar kembali. Snapshot menunjukkan catatan Handover: layanan checkout pulih, ID 1, sebagai data dummy. Contoh teks pada latihan boleh berbeda; ID bergantung data yang sudah ada. Teks UI saja perlu dilengkapi bukti API/SQL dan persistensi di langkah berikutnya.

6.1 Bandingkan respons API dengan baris SQL

Pada terminal yang sama, jalankan pemeriksaan baca saja:

Command yang dapat disalin

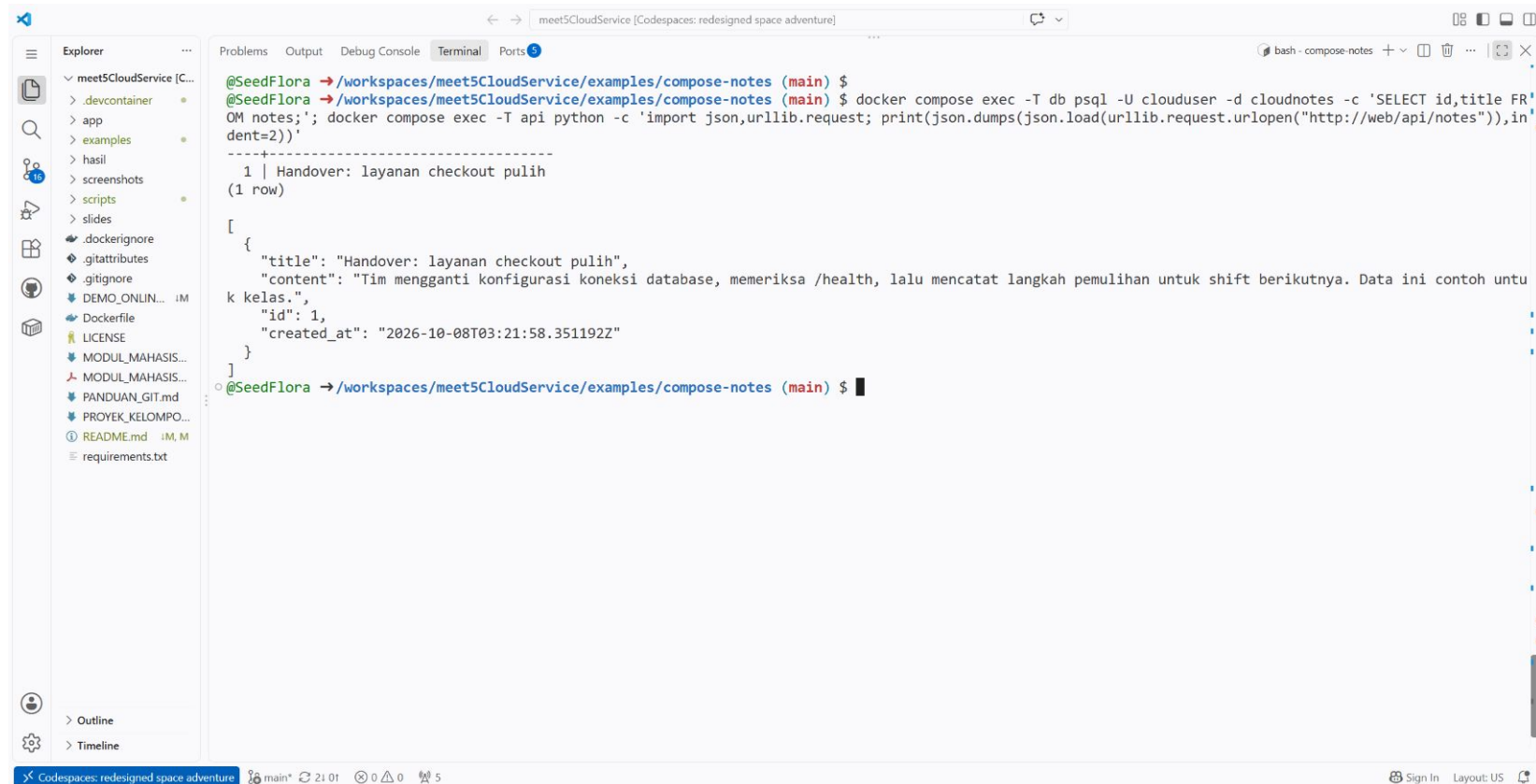
```
docker compose exec -T db psql -U clouduser -d cloudnotes \  
-c 'SELECT id, title FROM notes ORDER BY id;'\  
docker compose exec -T api python -c \  
'import json, urllib.request; print(json.dumps(json.load(urllib.request.urlopen("http://web/api/notes")), indent=2))'
```

Fungsi: command pertama membaca tabel pada PostgreSQL menggunakan `psql` di container database. Command kedua menjalankan Python di container API untuk melakukan GET melalui service `web`; hasilnya memperlihatkan JSON yang disajikan Nginx → FastAPI → PostgreSQL. Keduanya memakai program dalam container, sehingga Python atau PostgreSQL client tidak perlu dipasang pada host Codespaces. User/database di sini mengikuti nilai contoh; sesuaikan bila sengaja mengubah `.env`.

Kunci membaca: ID dan judul pada SQL harus sesuai dengan data pada JSON. Ini menghubungkan form browser dengan penyimpanan sebenarnya. Jangan mengubah tabel untuk sekadar membuat hasil cocok, dan jangan memakai SQL untuk menghapus catatan yang sedang dipakai demo persistensi.

Gambar 9. Baris PostgreSQL dan JSON API menampilkan catatan handover yang sama

```
docker compose exec -T db psql -U clouduser -d cloudnotes -c 'SELECT id,title FROM notes;'  
docker compose exec -T api python -c 'import json,urllib.request; print(json.dumps(json.load(urllib.request.urlopen("http://web/api/notes")),indent=2))'
```



Command aktual: `docker compose exec -T db psql -U clouduser -d cloudnotes -c 'SELECT id,title FROM notes;'`, lalu `docker compose exec -T api python -c 'import json,urllib.request; print(json.dumps(json.load(urllib.request.urlopen("http://web/api/notes")),indent=2))'`. Fungsi: membuktikan catatan pada UI memiliki baris database dan dapat dibaca melalui proxy web. Cara kerja: psql SELECT baca saja; Python di container API GET melalui web/proxy dan menata JSON. Baca hasil: SQL satu baris ID 1/judul Handover: layanan checkout pulih; JSON mempunyai ID/judul yang sama. Timestamp dicatat database; waktu browser dapat mengikuti zona pengguna.

6.2 Ulangi pemeriksaan dengan smoke script

Dari folder `examples/compose-notes`, dengan tiga service sudah berjalan:

```
bash scripts/smoke.sh
```

Script menjalankan Python **di container API** dan menggunakan URL internal `http://web/api`. Jadi request melewati service `web/proxy` → `API` → `PostgreSQL` tanpa bergantung pada Python di host atau port forwarding browser. Gunakan ini untuk memeriksa fungsi sebelum anggota tim mendemokan rilis; ini latihan bersama, bukan tugas atau penyerahan tambahan.

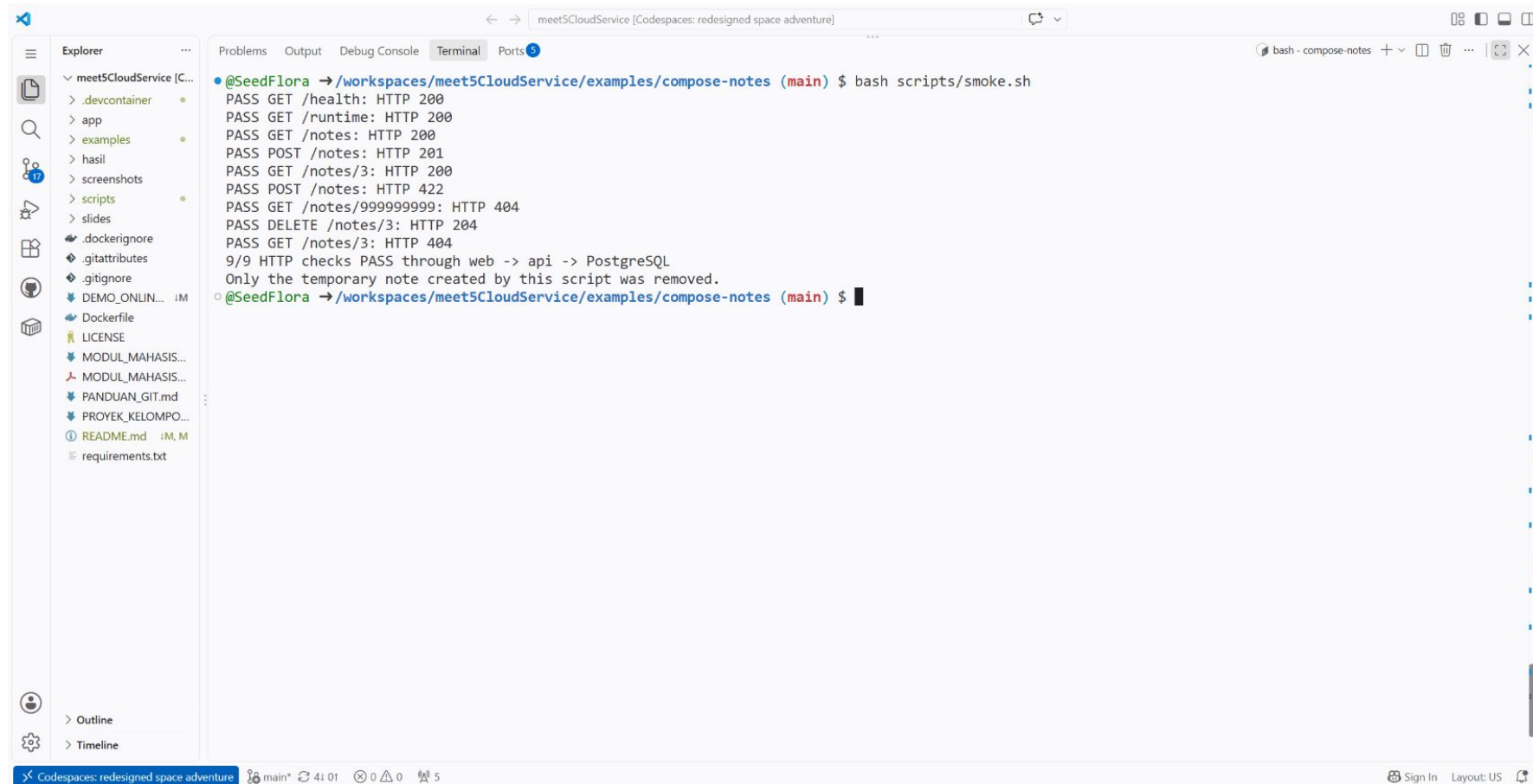
Urutan	Request	Status yang diharapkan	Yang diperiksa
1	GET <code>/health</code>	200	Respons menyatakan database <code>postgres</code> ; endpoint menguji koneksi database.
2	GET <code>/runtime</code>	200	Storage <code>postgres</code> dan proses Python diperiksa berjalan sebagai UID 10001 di container API.
3	GET <code>/notes</code>	200	Daftar dapat dibaca melalui proxy web.
4	POST <code>/notes</code> , judul Temporary smoke test	201	Membuat satu catatan dummy sementara dan menyimpan ID respons.
5	GET <code>/notes/{ID sementara}</code>	200	Catatan baru dapat dibaca; judul sesuai data uji.
6	POST <code>/notes</code> , judul kosong	422	Backend menolak input yang tidak valid.
7	GET <code>/notes/999999999</code>	404	ID yang tidak dibuat pada data demo ini ditolak sebagai tidak ditemukan.
8	DELETE <code>/notes/{ID sementara}</code>	204	Hanya catatan sementara milik script dihapus.
9	GET <code>/notes/{ID sementara}</code>	404	Catatan yang baru dihapus tidak lagi dapat dibaca.

Cara kerja `request(method, path, expected, payload=None)`: fungsi menggabungkan path dengan URL internal, mengubah payload menjadi JSON bila ada, lalu mengirim request dengan method dan header `Content-Type: application/json`. `urlopen` diberi timeout 10 detik. Python biasanya melempar `HTTPError` untuk status 4xx; script menangkap status/body tersebut agar **422 atau 404 yang sengaja diharapkan tetap dapat diuji**. Assertion membandingkan status aktual dengan status yang diharapkan. Body kosong pada 204 dibaca sebagai `None`, bukan dipaksa menjadi JSON. Kegagalan koneksi, timeout, status yang salah, atau assertion isi respons membuat command gagal dan perlu diperiksa; jangan menganggapnya hasil PASS.

Cara kerja try/finally: setelah POST berhasil, script menyimpan `created_id`. Bagian `finally` mencoba DELETE terhadap ID itu dan mengecek GET sesudahnya, termasuk bila pemeriksaan setelah pembuatan gagal. Ini menjaga catatan handover yang sudah ada karena script tidak menghapus seluruh tabel atau memakai ID catatan dosen. Cleanup dapat gagal bila layanan sedang tidak tersedia; bila itu terjadi, baca error dan periksa catatan sementara sebelum mencoba ulang. Ringkasan akhir hanya dicetak setelah alur serta assertion selesai; satu baris PASS status sebelum assertion isi bukan bukti seluruh script selesai.

Gambar 10. Smoke script melakukan sembilan pemeriksaan HTTP dan hanya menghapus catatan uji

```
bash scripts/smoke.sh
```



```
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $ bash scripts/smoke.sh
PASS GET /health: HTTP 200
PASS GET /runtime: HTTP 200
PASS GET /notes: HTTP 200
PASS POST /notes: HTTP 201
PASS GET /notes/3: HTTP 200
PASS POST /notes: HTTP 422
PASS GET /notes/999999999: HTTP 404
PASS DELETE /notes/3: HTTP 204
PASS GET /notes/3: HTTP 404
9/9 HTTP checks PASS through web -> api -> PostgreSQL
Only the temporary note created by this script was removed.
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $
```

Command: `bash scripts/smoke.sh`. Fungsi: memeriksa operasi baca/tulis, validasi, respons tidak ditemukan, dan penghapusan melalui proxy web. Hasil aktual pada Codespaces: 9/9 HTTP checks PASS through web → api → PostgreSQL. Catatan sementara pada snapshot mendapat ID 3 dan dihapus; ID dapat berbeda pada eksekusi lain. Catatan handover ID 1 tetap dipakai untuk uji persistensi. Hasil ini menguji alur internal service; keberhasilan tab Ports dan tampilan browser dibuktikan terpisah oleh screenshot frontend.

7. Buktikan persistensi: down, lalu up

Pastikan setidaknya satu catatan terlihat sebelum menghentikan service. Jangan menjalankan langkah ini pada service atau database di luar project demo.

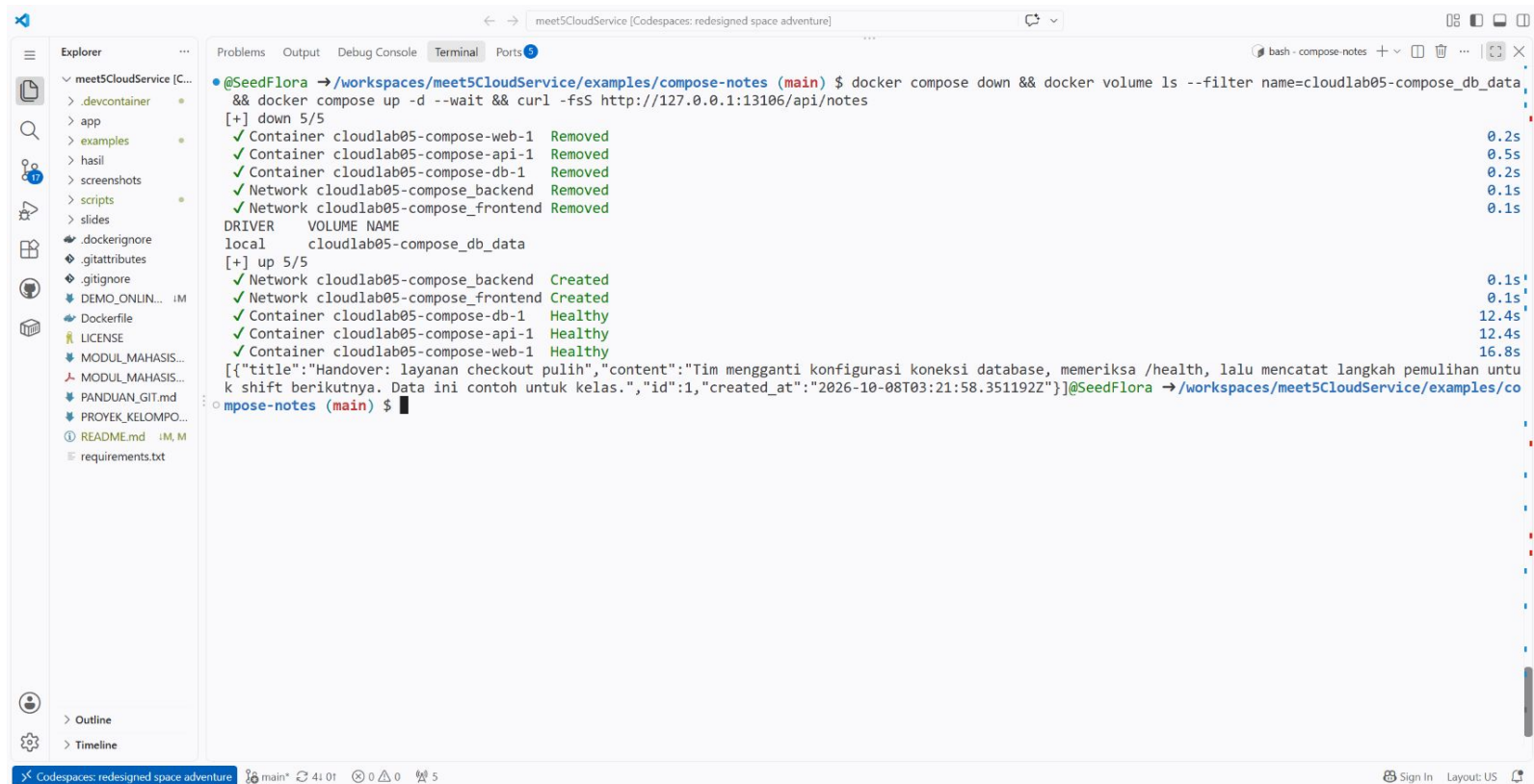
```
curl -fsS "http://127.0.0.1:${API_HOST_PORT}/notes"
docker compose down
docker volume inspect cloudlab05-compose_db_data
docker compose up -d --wait
curl -fsS "http://127.0.0.1:${API_HOST_PORT}/notes"
```

Cara kerja: `down` menghapus container dan network project, tetapi named volume dipertahankan. Saat service dibuat kembali dengan nama project yang sama, PostgreSQL menggunakan data pada volume yang sama. Catatan tetap muncul meskipun container baru dibuat; ini berbeda dari dict memori pada API utama Lab 05. Nama default project contoh ini adalah `cloudlab05-compose`, sehingga volume bernama `cloudlab05-compose_db_data`. Mengganti project name dapat membuat volume lain dan tampak seperti data hilang.

Kunci: hasil setelah up harus masih memuat catatan yang sudah dibuat. Jika data tidak muncul, periksa project name, volume yang dipakai, log database, dan apakah sebelumnya ada penghapusan volume atau catatan. Named volume menyimpan data pada mesin Docker itu; volume bukan backup, dan tidak otomatis disalin ke GitHub atau laptop ketika Codespace dihapus. [Volumes](#), [Compose down](#)

Gambar 11. Container dihapus dan dibuat lagi, tetapi volume dan catatan PostgreSQL tetap ada

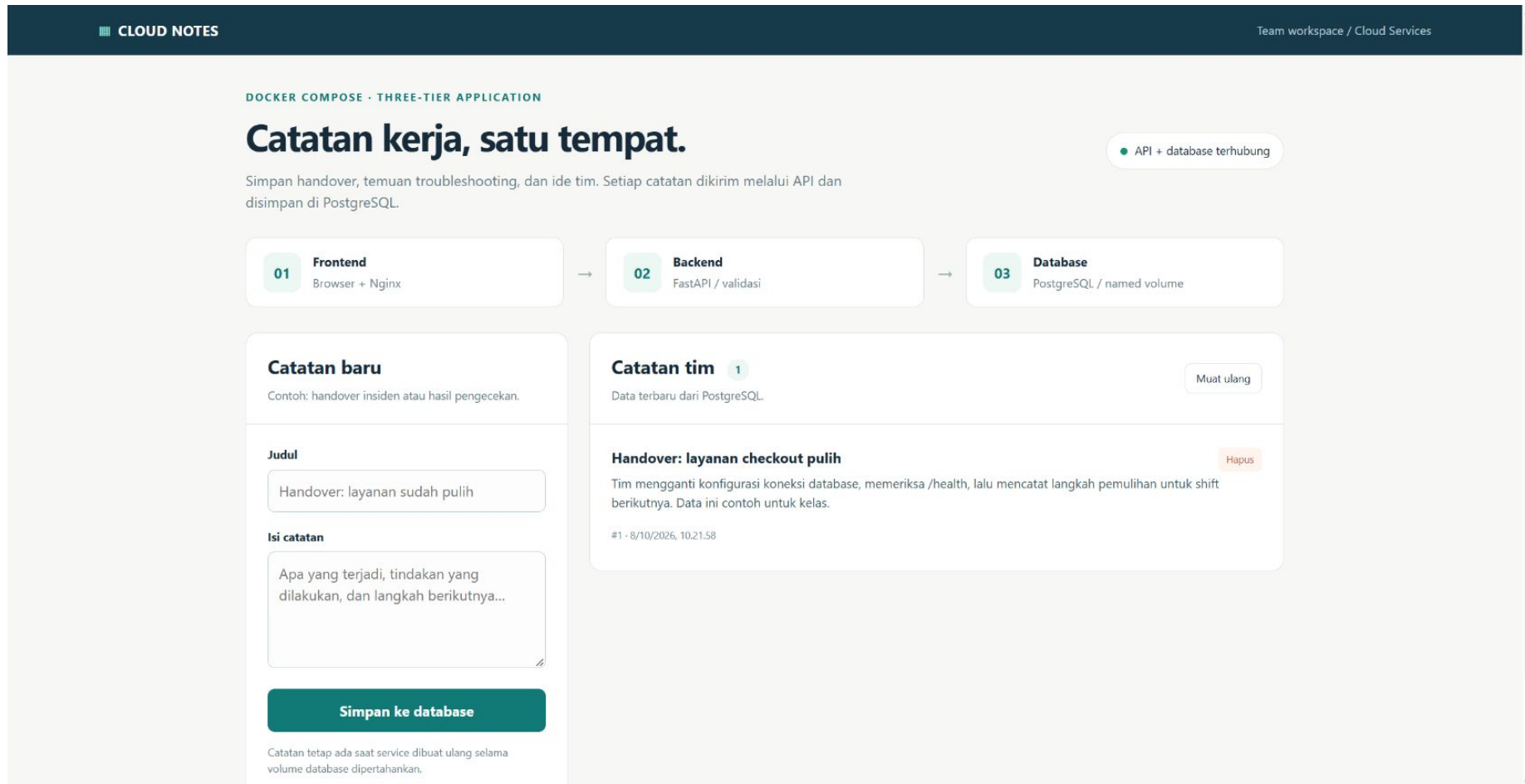
```
docker compose down
docker volume ls --filter name=cloudlab05-compose_db_data
docker compose up -d --wait
curl -fsS http://127.0.0.1:13106/api/notes
```



Command aktual: `docker compose down`, `docker volume ls --filter name=cloudlab05-compose_db_data`, `docker compose up -d --wait`, lalu `curl -fsS http://127.0.0.1:13106/api/notes`. Fungsi: membuktikan data bertahan setelah container/network dibuat kembali. Baca hasil: container Removed, named volume tetap terdaftar, tiga service kembali Healthy, dan JSON tetap berisi catatan handover ID 1. Panduan memakai `docker volume inspect` sebagai pemeriksaan volume yang lebih rinci; keduanya adalah pemeriksaan baca saja.

Gambar 12. Frontend setelah service dibuat kembali masih menampilkan catatan ID 1

setelah up selesai, buka kembali frontend port 13106 dan klik Muat ulang atau reload halaman.



Langkah: setelah up selesai, buka kembali frontend port 13106 dan klik Muat ulang atau reload halaman. Fungsi: memeriksa hasil persistensi dari sudut pengguna browser. Baca hasil: API + database terhubung, judul/isi handover dan ID 1 tetap muncul. Gabungkan bukti UI ini dengan down/up dan respons API di screenshot sebelumnya; tampilan yang tidak dimuat ulang saja belum membuktikan data bertahan.

8. Cleanup setelah demo

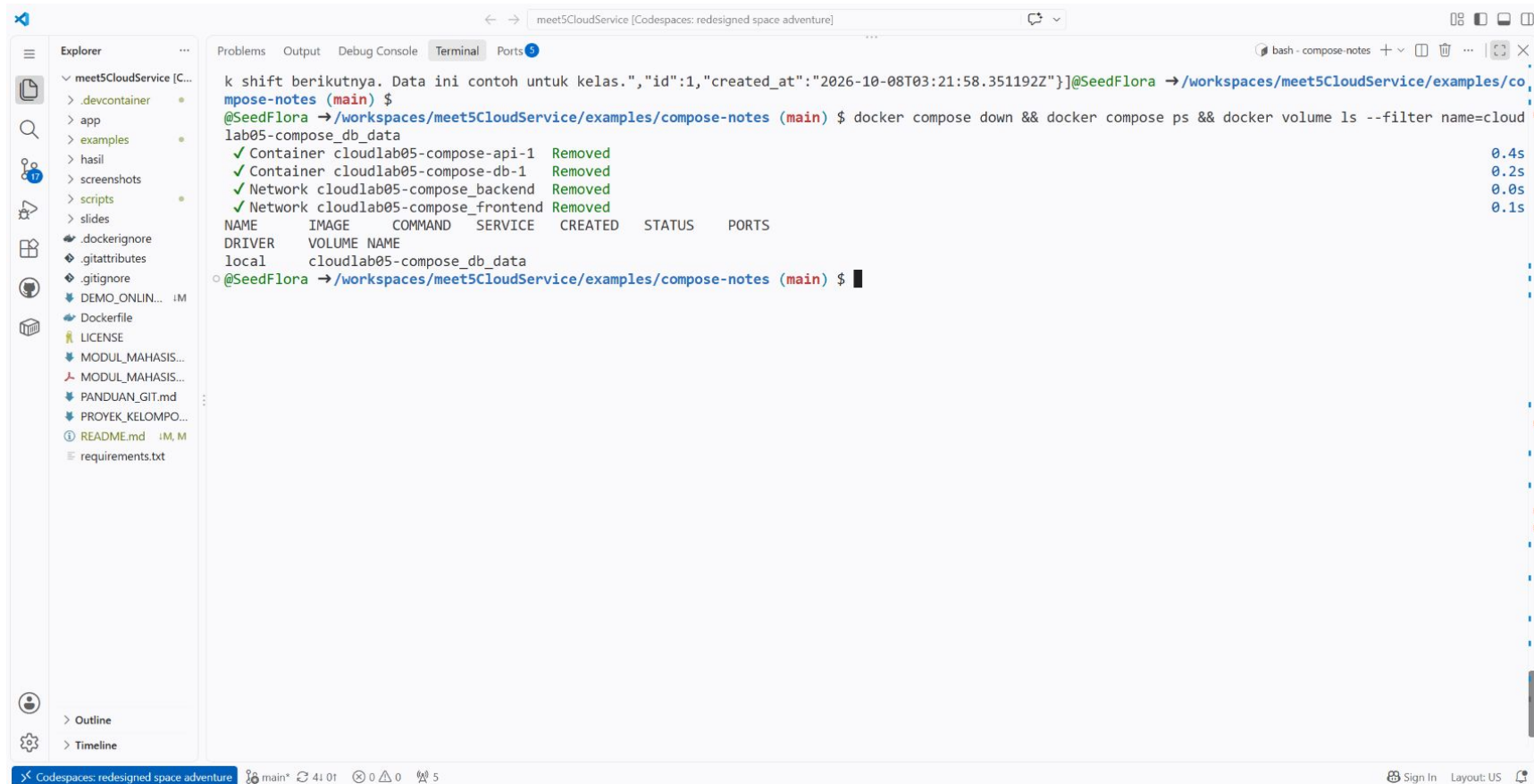
Dari folder `examples/compose-notes`:

```
docker compose down
docker compose ps
```

Ini menghentikan dan menghapus container/network **project demo**; named volume dan data PostgreSQL tetap ada untuk sesi berikutnya. Tabel status tidak lagi menunjukkan service yang aktif. Jangan menambahkan `-v` pada alur ini karena `docker compose down -v` menghapus named volume project dan data di dalamnya. Penghapusan volume hanya dilakukan bila dosen memang memutuskan reset data dummy; itu bukan langkah rutin untuk membuktikan persistensi.

Gambar 13. Cleanup menghapus container dan network demo, sedangkan named volume tetap ada

```
docker compose down
docker compose ps
docker volume ls --filter name=cloudlab05-compose_db_data
```

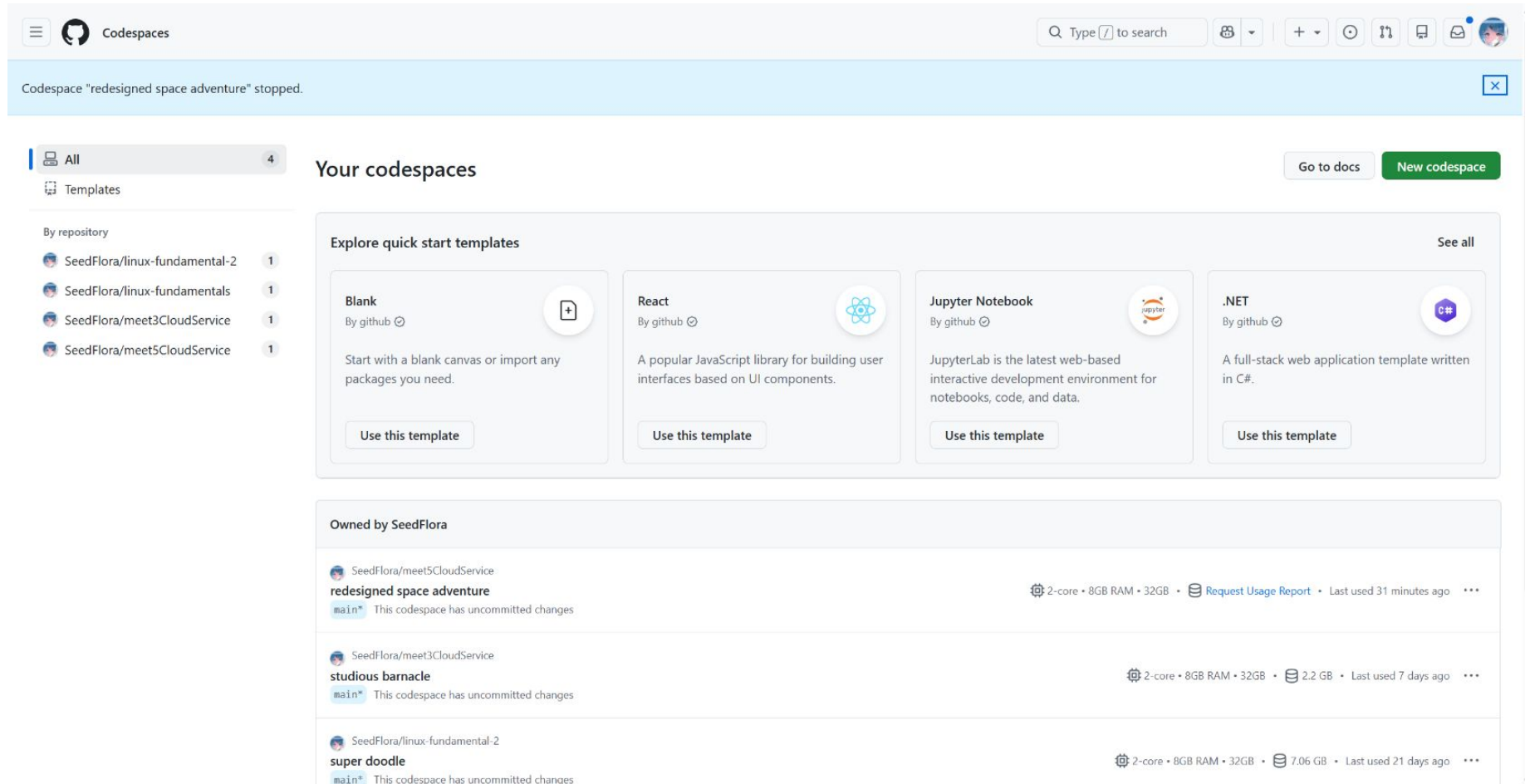


Command aktual: `docker compose down`, `docker compose ps`, dan `docker volume ls --filter name=cloudlab05-compose_db_data`. Fungsi: mengakhiri layanan demo tanpa menghapus data kelas. Baca hasil: container/network Removed, tabel ps kosong, dan volume `cloudlab05-compose_db_data` masih terdaftar. Ini belum menghentikan compute Codespaces; lakukan Stop codespace setelah demo online selesai.

Kembali ke root repo dengan `cd ../../`. Jika API memori dari demo sebelumnya masih berjalan, bersihkan dengan `bash scripts/demo-online.sh stop`. Di Codespaces, selanjutnya pilih **Stop codespace** pada halaman Codespaces agar compute berhenti. Storage tetap dihitung selama Codespace dan volumenya disimpan; menghapus Codespace membuang data lokal tersebut. Simpan kode melalui Git dan gunakan strategi backup yang sesuai bila data perlu dipertahankan.

Gambar 14. GitHub mengonfirmasi Codespace kelas sudah stopped

github.com/codespaces → menu ... pada lingkungan kelas → Stop codespace.



Langkah: github.com/codespaces → menu ... pada lingkungan kelas → Stop codespace. Fungsi: mengakhiri compute setelah seluruh Compose/Scout selesai. Bukti aktual: banner Codespace redesigned space adventure stopped. Storage tetap dihitung selama Codespace disimpan; Stop tidak sama dengan Delete. Perubahan belum di-commit yang terlihat tersimpan di lingkungan stopped, tetapi perlu commit/push kode atau backup data yang ingin dipertahankan sebelum lingkungan dihapus.

9. Kunci semua pertanyaan diskusi

Pertanyaan	Kunci jawaban
Mengapa Compose setelah Dockerfile?	Dockerfile mendefinisikan pembuatan image; Compose mendefinisikan cara beberapa service dijalankan bersama dengan network, environment, dan volume.
Mengapa API menghubungi <code>db</code> , bukan localhost?	Localhost pada container API menunjuk container API. Nama service <code>db</code> ditemukan oleh DNS network Compose dan mengarah ke container database.
Mengapa browser tidak membuka <code>http://api:8000</code> ?	Browser laptop berada di luar network Compose. Browser membuka port host/URL forwarding, lalu Nginx menghubungi service internal <code>api</code> .
Apa manfaat frontend memakai <code>/api/*</code> relatif?	Request mengikuti origin frontend; Nginx meneruskannya ke backend. Kode browser tidak perlu menebak URL Codespace atau alamat IP container.
Apa beda port frontend, API, dan database?	Default 13006 adalah port host frontend, 13005 port host API, dan 5432 port internal PostgreSQL. Screenshot online memakai host 13106/13105; container tetap web 80/API 8000. Database tidak dipublikasikan ke host pada demo.
Mengapa <code>depends_on</code> dan healthcheck?	Dependency kesiapan membantu menghindari API dimulai sebelum database siap. Aplikasi tetap harus menangani gangguan selama runtime; healthcheck bukan bukti semua fitur bekerja.
Mengapa data bertahan setelah down/up?	PostgreSQL menulis ke named volume yang dipakai kembali oleh container database pada project yang sama. Dict memori pada API utama tidak memiliki perilaku ini.
Apakah <code>down -v</code> sama dengan <code>down</code> ?	Tidak; <code>-v</code> juga menghapus volume project. Data dummy di volume tersebut hilang. Demo utama memakai <code>down</code> tanpa <code>-v</code> .
Apakah volume otomatis menjadi backup GitHub?	Tidak. Volume berada pada mesin Docker. Commit/push kode tidak menyertakan isi database; kehilangan mesin/Codespace membutuhkan backup yang sesuai.
Apakah password contoh aman untuk sistem nyata?	Tidak. Password sengaja diketahui untuk kelas. Gunakan secret tersendiri dan pengelolaan secret pada sistem nyata; jangan memublikasikan kredensial nyata.
Build sukses dan tiga service healthy: boleh langsung rilis?	Masih perlu tes create/read/validasi, pemeriksaan konfigurasi/secret/akses, konteks risiko dependency, serta prosedur rilis. Scout membantu peninjauan paket, bukan menggantikan tes aplikasi.
Mengapa respons 422 dan 404 dihitung PASS pada smoke script?	Script sengaja mengirim input tidak valid atau ID yang tidak ada. PASS berarti status sesuai kontrak untuk kasus itu; semua request tidak harus menghasilkan 200.
Mengapa memakai <code>finally</code> dan ID dari respons POST?	Agar cleanup mencoba menghapus catatan sementara yang benar meskipun tes berikutnya gagal, tanpa menebak ID atau menghapus catatan dosen. Jika koneksi gagal, cleanup tetap perlu diperiksa.
Apakah smoke script memerlukan Python host dan membuktikan forwarding browser?	Python dijalankan dalam container API, jadi host tidak perlu Python. URL internal web menguji proxy dan backend/database; forwarding dan tampilan browser diuji melalui tab Ports/frontend secara terpisah.

10. Kendala umum

Gejala	Pemeriksaan
Port 13005/13006 dipakai aplikasi lain	Baca <code>docker ps</code> ; jangan menghentikan service lain sembarangan. Gunakan lingkungan kelas terpisah atau ubah mapping dengan sadar dan sesuaikan URL/forwarding.
Config mengeluh variabel belum tersedia	Pastikan <code>.env</code> disalin di folder <code>examples/compose-notes</code> , lalu <code>docker compose config --quiet</code> .
API unhealthy atau tidak dapat menghubungi database	Baca <code>docker compose logs --tail 30 api</code> dan <code>db</code> ; periksa kesiapan database, environment, serta nama service <code>db</code> .
<code>nslookup api</code> mencoba suffix cloud atau menghasilkan NXDOMAIN tambahan	Gunakan <code>docker compose exec -T web nslookup api</code> . untuk query absolut. DNS service dan aplikasi tetap memakai <code>api/db</code> ; jangan menulis IP hasil lookup sebagai konfigurasi tetap.
Frontend terbuka tetapi API gagal	Uji API langsung pada port host API, lalu <code>/api/health</code> pada port host web; gunakan pasangan 13005/13006 atau 13105/13106 sesuai <code>.env</code> . Periksa log <code>web</code> dan <code>api</code> .
<code>localhost</code> browser tidak membuka layanan online	Gunakan URL tab Ports Codespaces, bukan localhost laptop.
Data hilang setelah mengganti konfigurasi	Periksa project name dan volume. Perubahan environment password tidak otomatis mengubah user/password database yang sudah terinisialisasi pada volume lama.
<code>--wait</code> tidak dikenali	Periksa versi Compose. Jalur kelas memakai plugin Compose modern; pada lingkungan lama, tunggu healthcheck dan tes HTTP secara eksplisit atau gunakan lingkungan Codespaces yang disiapkan.
<code>bash scripts/smoke.sh</code> tidak menemukan API atau koneksi gagal	Jalankan dari <code>examples/compose-notes</code> setelah <code>up/healthcheck</code> selesai; periksa <code>ps</code> dan <code>logs</code> . Script memakai Python container, sehingga memasang Python pada host tidak memperbaiki API yang mati.

11. Sumber resmi

- [Compose application model, networking, dan startup order](#).
- [Compose config, up, ps, logs, dan down](#).
- [Volumes dan Compose environment interpolation](#).